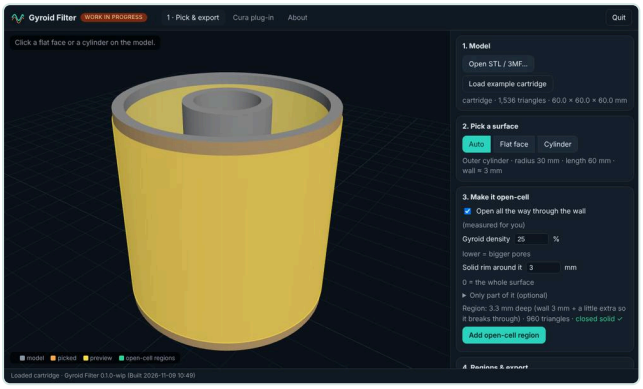
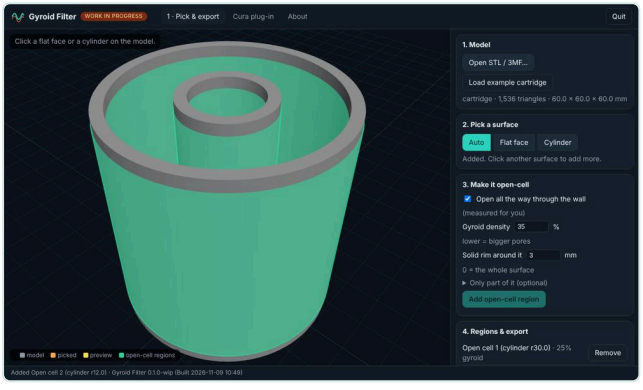
Gyroid Filter

Pick a flat face or a cylinder and make it open-cell gyroid, for fillable carbon filters. Cura, PrusaSlicer, OrcaSlicer.

WORK IN PROGRESS · MANY FEATURES MAY BE BROKEN

Version 0.1.0-wip · built 2026-11-09 10:49





Source-available, non-commercial (PolyForm Noncommercial 1.0.0). Provided as is, with no warranty and no liability. Not a medical or safety device; nothing here has been tested for filtration performance.

Contents

1. Readme · Overview and how to use it
2. How It Works · How it works
3. Design Notes · Design notes and research
4. Changelog · Changelog
5. Third Party · Third-party parts
6. License · Licence

Readme

Overview and how to use it

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN · proof of concept · version 0.1.0-wip

> Source-available, **non-commercial** (PolyForm Noncommercial 1.0.0). Provided as is, **no warranty and no liability**. Not a medical or safety device: check that a printed filter suits what you filter. PLA and PETG are not rated for food, chemicals or heat.

📘 The whole manual and research notes as one PDF

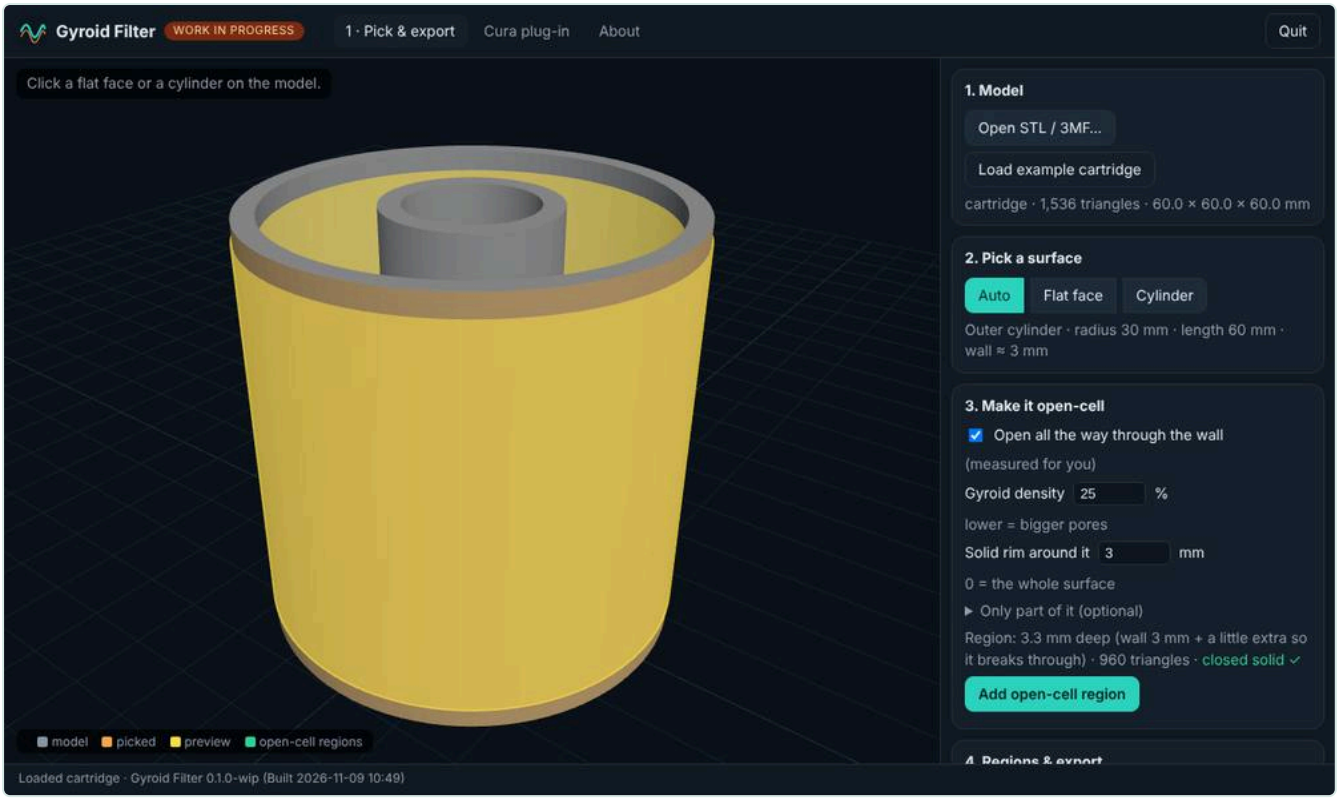
Gyroid Filter

Pick a **flat face** or a **cylinder** on a model and make part or all of it **open-cell gyroid**, for printing **fillable carbon filters**. The gyroid replaces the solid wall, so air can pass through while the carbon granules stay inside.

It works with **Cura**, **PrusaSlicer** and **OrcaSlicer**:

Slicer	How you use it
Cura	A plug-in adds a *Gyroid Filter* tool. Select the model, pick the tool, click a face or cylinder.
PrusaSlicer	Pick in the app, press *PrusaSlicer 3MF*, open the file. The open-cell regions are already there as modifiers.
OrcaSlicer	Same, with *OrcaSlicer 3MF* (see what is tested).

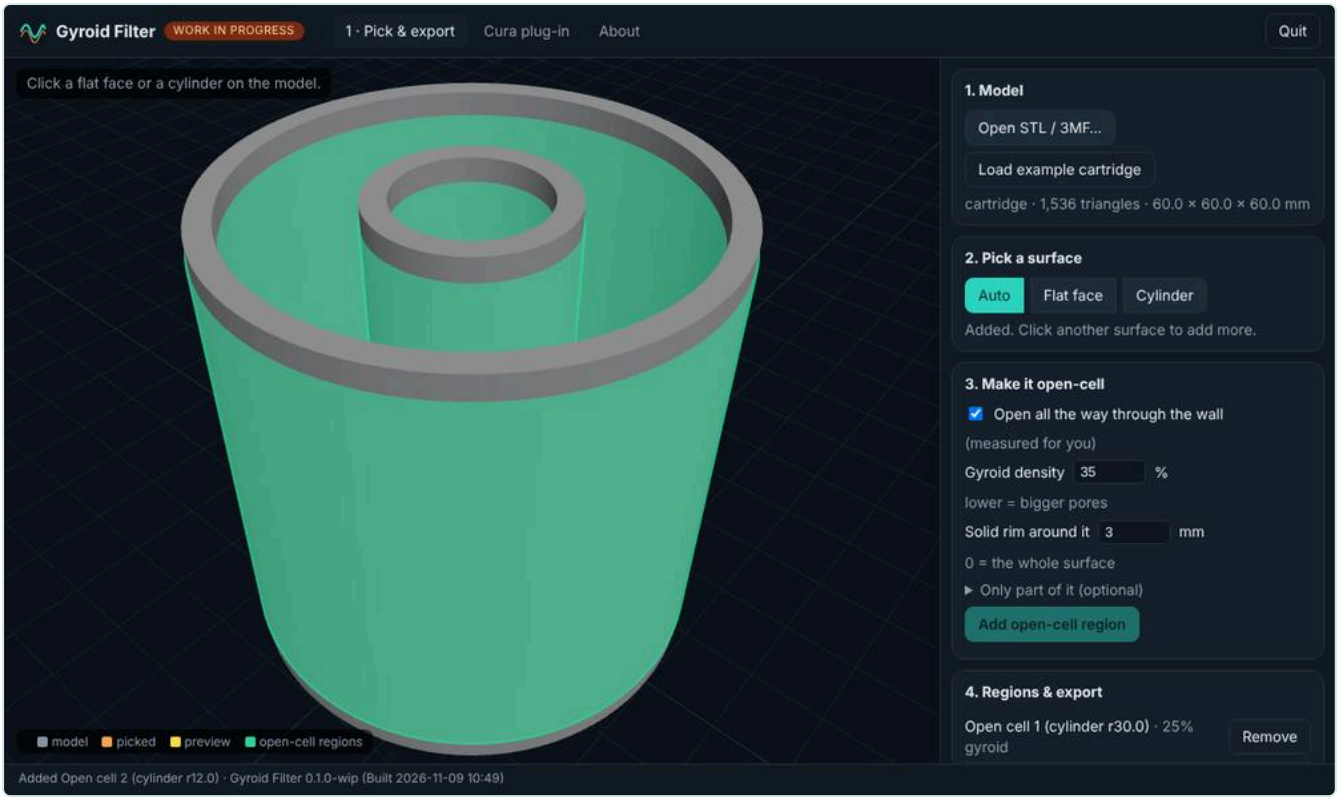
PrusaSlicer and OrcaSlicer have no plug-in system that can pick faces, so for them the companion app does the picking and hands the slicer a normal 3MF. Your original model is never changed.



Quick start (no terminal needed)

1. Unzip, and **double-click** the launcher for your computer: - Linux: `GyroidFilter-Linux.sh` (choose *Run* or *Run in Terminal*) - Windows: `GyroidFilter-Windows.bat` - Mac: `GyroidFilter-Mac.command` (first time: right-click, then *Open*, because it is unsigned)
2. Your browser opens. Press **Load example cartridge** (or open your own STL/3MF).
3. **Click** a face or a cylinder on the model. The picked surface turns orange; a yellow preview shows the region that will become open-cell.
4. Press **Add open-cell region**. Repeat for other surfaces.
5. Press **PrusaSlicer 3MF**, **OrcaSlicer 3MF** or **Region STLs**, and open the file in your slicer. For Cura see below.

Needs Python 3.8 or newer (nothing else; no packages to install). The launcher says what to do if Python is missing.



What the options mean

Option	What it does
Open all the way through the wall (default)	Measures the wall thickness under the picked surface and makes the region that deep, plus 0.3 mm so it breaks through. For a filter wall this is what lets air cross. Untick to choose a depth.
Gyroid density (5–60 %)	The slicer's infill density inside the region. Lower means bigger pores. How big the pores are for a given density is not measured yet: print a small test piece with your nozzle and your carbon before you commit.
Solid rim (mm)	Leaves a solid frame around the open area (for a cylinder: at both ends) so the part keeps its strength and has somewhere to glue or seal. 0 = the whole surface.
Only part of it	Limits the region to a window on a flat face (rectangle or oval, in % of the face), or to a stretch along and around a cylinder.

"Open-cell" here means the region is printed with **no walls, no top or bottom skin and no "small area becomes solid" rule**, so the gyroid strands *are* the surface.

The settings it applies

	Cura (cutting mesh)	PrusaSlicer (modifier)	OrcaSlicer (modifier)
Pattern	`infill_pattern` = gyroid	`fill_pattern` = gyroid	`sparse_infill_pattern` = gyroid
Density	`infill_sparse_density`	`fill_density`	`sparse_infill_density`
Walls	`wall_line_count` = 0	`perimeters` = 0	`wall_loops` = 0

	Cura (cutting mesh)	PrusaSlicer (modifier)	OrcaSlicer (modifier)
Top/bottom skin	`top_layers`, `bottom_layers` = 0	`top_solid_layers`, `bottom_solid_layers`, `*_solid_min_thickness` = 0	`top_shell_layers`, `bottom_shell_layers`, `*_shell_thickness` = 0
Small areas	`min_infill_area` = 0	`solid_infill_below_area` = 0	`minimum_sparse_infill_area` = 0

If a 3MF does not do what you want in your slicer, you can do the same by hand: add the **Region STLs** as modifiers and set these five things.

Cura

1. In the app open the **Cura plug-in** tab, press **Install** next to your Cura version, restart Cura.
2. In Cura: select your model, choose the **Gyroid Filter** tool in the left toolbar, and click a face or cylinder.
3. The region appears as a child of the model (a *cutting mesh* carrying the settings above). **Slice**, then look at the layer view.
4. Change density or rim in the tool panel and press **Re-apply to last pick**; **Remove gyroid regions** clears them.

Cura's log (`cura.log`) gets a line `GyroidFilter: core self-test PASS` when the plug-in loads; the Cura tab in the app reads it and shows a green tick.

What is tested, and what is not

Part	Status
Face and cylinder detection, region solids, thickness probe	✅ unit tests on the example cartridge, boxes, a half-pipe and a finely tessellated cylinder
PrusaSlicer 3MF	✅ sliced in a real PrusaSlicer 2.7.2: inside the region only gyroid-style infill, no perimeters; above the rim normal walls; flat oval window in a plate also checked
OrcaSlicer 3MF	⚠️ **not opened in Orca yet.** Written to Orca's own layout by reading Orca's importer source and its setting names. If it misbehaves, use the Region STLs route
Cura plug-in	✅ **Tested by the author in a real Cura on Linux Mint: it works well.** Also tested against stand-ins for Cura's API, and every Cura setting name is checked against Cura's `fdmprinter.def.json`. Windows and Mac are not tested yet. Please report what happens on your setup
Web app	✅ driven in headless Chrome (pick, preview, add, export, Cura tab): no JavaScript errors
Linux launcher	✅ run for real. **Windows and Mac launchers were not run**
Printing a filter	❌ nothing has been printed or filtered with this yet. Pore size versus density is unmeasured
7th-gen Core i3	⚠️ not measured on one. On the 2-core test machine a 200,000-triangle model loads and picks in about a second and needs about 200 MB; see DESIGN_NOTES.md

Known limitations

- Picks **flat faces and true cylinders** (holes and outer surfaces, full or partial arcs). Cones, spheres and free-form surfaces are not recognised.
- It does not design the filter housing, the fill port or the lid. It only makes the chosen walls open-cell.
- 3MF files are read for geometry only; modifiers and settings inside them are ignored. Open your original STL, or export STL from your CAD.
- Both slicers still print a thin boundary wall where the open-cell region meets the rest of the part (inside the part, never on the surface you picked).
- The 3MF is placed on a default bed centre (Prusa 125,105; Orca 128,128). If it sits off your bed, press **A** (arrange) in the slicer or set the bed centre in the app.
- Very big models are refused with a message if they will not fit in free memory.

Troubleshooting

Problem	What to try
Launcher says Python is missing	Install Python 3.8+ (the message tells you how), then double-click again
Browser did not open	Copy the `http://127.0.0.1:…` address printed in the launcher window
Click does nothing	Wait for the *Finding the surface* box to finish; on a big model it can take a few seconds
"No cylinder found"	You chose *Cylinder* on something that is not one; use *Auto* or *Flat face*
Region shows **NOT closed**	The picked surface has holes or overlaps; pick a neighbouring surface
Orca 3MF opens as one solid	Use **Region STLs** and add them as modifiers by hand
Cura tab shows no Cura	Start Cura once so it creates its folder, then press *Look for Cura again*
Cura loads it but shows no tool	Open `cura.log` and look for `GyroidFilter`; please report the line

More

- HOW_IT_WORKS.md: what it reads, changes and never does
- DESIGN_NOTES.md: research notes, decisions, rejected ideas, measurements
- CHANGELOG.md · docs/HISTORY.md · THIRD_PARTY.md · CONTRIBUTING.md · SECURITY.md

Related projects by the same author: flatlock / Shift-LoK (interlocking walls for the same three slicers) and cura-hybrid-supports (another Cura plug-in). This project shares their approach of working inside the slicer's own settings rather than rewriting G-code.

Project started 2026-11-09 09:55. Built by Matthew Armstrong with Claude.

How It Works

How it works

In one paragraph

You click a surface. The app finds the whole flat face or cylinder that click belongs to, measures how thick the material is under it, and builds a closed 3D "region" volume under that surface. The region is not part of your model: it is handed to the slicer as a **modifier** (PrusaSlicer, OrcaSlicer) or **cutting mesh** (Cura), carrying settings that print the overlapping part as gyroid with no walls and no skin. The slicer does all the real work.

What it reads

- The STL or 3MF you open (geometry only). Nothing is uploaded anywhere: the app is a small web server that listens **only on your own computer** (`127.0.0.1`) on a random port, and only answers requests carrying a secret token made at start-up.
- For the Cura tab: the list of Cura settings folders in your home directory (to offer *Install*) and the last lines of `cura.log` mentioning GyroidFilter.

What it changes

- Exports are new files that you save from the browser. Your original model file is never touched.
- *Install* in the Cura tab copies the plug-in folder to `…/cura/<version>/plugins/GyroidFilter/` and reads it back to check. *Uninstall* removes only that folder.
- In Cura, the plug-in adds child nodes named "Gyroid region" under the model you clicked. Removing them (button, or Cura's normal delete) puts the model back as it was.

What it never does

- It does not rewrite or post-process G-code.
- It does not change your slicer's profiles or settings files.
- It does not connect to the internet.

Steps in detail

1. **Topology.** Triangles that share an edge are linked. Vertices that coincide (to 0.0001 mm) are welded.
2. **Flat face.** From the clicked triangle, neighbours are added while their normal is within 1° of the first and their corners within 0.05 mm of its plane.
3. **Cylinder.** Around the click, triangles are gathered until their normals have turned about 30°. The cylinder axis is the one direction all those normals are perpendicular to (smallest eigenvector of their covariance); a circle is fitted in the plane across the axis; then every connected triangle whose corners lie on that circle and whose normal is perpendicular to the axis is added. The length and the covered angle (full circle or an arc) come from the result. Whether material is inside (outer surface) or outside (hole) comes from which way the normals point.

4. **Auto** prefers the cylinder if it covers more triangles than the flat patch, because a finely tessellated cylinder looks flat over a few facets.
5. **Thickness.** Up to three rays from the surface into the material; the median distance to the far side is the wall thickness.
6. **Region.** A flat face becomes a prism (the face's triangles pushed to the chosen depth, clipped to the window if any). A cylinder becomes a ring segment between two radii. Both stick out 0.3 mm beyond the surface so they cut cleanly through the skin. The app checks each region is a closed solid and shows ✓ or a warning.
7. **Export.** The region is added to the model as a second volume with the open-cell settings (see the table in the README).

Why open-cell gyroid helps a filter

Gyroid infill is a continuous wavy sheet that splits space into two interlocking channels. With no walls or skin, those channels reach the outside, so air can flow through a wall of it while the strands hold granules back. Pore size is set by density and nozzle width; the app does not predict it.

Design Notes

Design notes and research

Everything learned while building Gyroid Filter 0.1.0-wip, so nobody (including future me) has to redo it. Facts marked **verified** were checked by running something; **from source** means read in a slicer's source code but not run; **unknown** means exactly that.

The request

A plug-in for Cura, PrusaSlicer and OrcaSlicer that lets you **select a face or cylinder** and make part or all of it **open-cell gyroid**, for **fillable carbon filters**.

Findings

The slicer-native way: a volume with its own settings

All three slicers can print one region of a part with different infill/wall settings, using a second overlapping volume:

Slicer	Name	Per-volume settings
PrusaSlicer	modifier	yes: any per-object setting (pattern, density, perimeters, skin...)
OrcaSlicer	modifier part	yes, same idea with Orca's setting names
Cura	**cutting mesh** (`cutting_mesh`)	yes: the overlap is printed with the cutting mesh's own settings

Cura's own description of `cutting_mesh` (from `fdmprinter.def.json`): limit the volume of this mesh to within other meshes, to print some areas with different settings. The related `infill_mesh` only replaces *infill* and its description suggests keeping one wall and no skin, so the part's own walls would stay on the surface; that is why the plug-in uses the cutting mesh and not the infill mesh. **From source / description, not run.**

Why the region must have zero walls and zero skin

If the region kept walls or skin, the surface you picked would stay closed. Inside the region the app sets walls = 0, top/bottom layers = 0, minimum solid thickness = 0 and "small area becomes solid" = 0. **Verified in PrusaSlicer 2.7.2**: sliced G-code shows only `Internal infill` in the wall bands, no `Perimeter` / `External perimeter` / `Solid infill`; the infill segment directions are many-valued even though the test's global pattern was straight lines, so the gyroid override is in effect.

PrusaSlicer 3MF layout (verified)

One `<object>` holding every volume's triangles back to back; `Metadata/Slic3r_PE_model.config` lists `<volume firstid lastid>` ranges with `volume_type` = `ModelPart` or `ParameterModifier` and one `<metadata type="volume" key=… value=…/>` per setting. The item transform places the object. PrusaSlicer 2.7.2 reads this and slices it.

OrcaSlicer 3MF layout (from source, **not run**)

Read from `src/libslic3r/Format/bbs_3mf.cpp` (OrcaSlicer main):

- Orca reads **only** `Metadata/model_settings.config` for part settings; the line that would read PrusaSlicer's `Slic3r_PE_model.config` is commented out. So a PrusaSlicer-style 3MF opens in Orca with the modifier merged into the body as solid geometry. The two flavours must differ.
- Volumes are built from **sub-objects** (`_generate_volumes_new`): each part is its own `<object>` with a mesh, and one assembly `<object>` lists them as `<component objectid=…>`. `<part id=… subtype="modifier_part">` in `model_settings.config` points at the sub-object id.
- Subtype strings: `normal_part`, `negative_part`, `modifier_part`, `support_blocker`, `support_enforcer`.
- Every other `<metadata key value>` of a part goes to `config.set_deserialize(key, value)`, i.e. Orca's own setting names (`sparse_infill_pattern` etc.); all names used were checked to exist in Orca's `PrintConfig.cpp`.
- A file without the `Application` = Orca/Bambu tag is treated as "from another vendor"; the model settings file is still read. The exporter deliberately does not claim to be Orca-made.
- **Unknown:** whether Orca 2.3.x really shows the modifier with those settings when this exact file is opened. First thing to test when an Orca is at hand. Fallback shipped: Region STLs + the settings table.

Cura plug-in (from source, not run)

- Pattern copied from Cura's own `SupportEraser` tool: `Tool` subclass, scene node with `SettingOverrideDecorator` (made by `CuraSceneNode`), `SettingInstance` per setting, `GroupedOperation` of `AddSceneNodeOperation` + `SetParentOperation` + `TranslateOperation`.
- Tool panel: `"tool_panel": "GyroidFilterPanel.qml"` in `getMetaData`; QML talks to the tool through `UM.ActiveTool.properties.getValue(name)`, `UM.Controller.setProperty(name, value)` (calls `set<Name>`), and `UM.ActiveTool.triggerAction("method")` (calls the method).
- Picking: the selection render pass tells which model was clicked; `Camera.getRay(x, y)` gives the ray; the plug-in intersects it with that model's world-space triangles (numpy, Möller–Trumbore) to learn the *triangle*, which the render pass alone cannot say.
- Plug-in API: `plugin.json` has `"api": 8` and `supported_sdk_versions`; Uranium accepts a plug-in whose major version equals Cura's and whose minor is not higher.
- The geometry core is copied into the plug-in by `tools/build_cura_plugin.py` (single source of truth in `src/gyroid_filter/`); a test fails if the copy is stale.
- Start-up self-test writes `GyroidFilter: core self-test PASS` to Cura's log. The installer window reads it back, so "installed" is never reported on a guess.

Gyroid facts used

Gyroid is available as an infill pattern in all three (`gyroid`, `fill_pattern`, `sparse_infill_pattern`). It is open in both directions (no enclosed pockets) which is why it suits flow-through. Pore size depends on density and line width; **not measured here**.

Decisions, with the rejected alternatives

Decision	Why	Rejected
Modifier / cutting-mesh volumes carrying settings	Slicer does the work; works with each slicer's own profile; no G-code surgery; survives slicer updates	**G-code post-processing**: cannot know which extrusions belong to a face; would have to delete walls and invent gyroid
Companion app + 3MF for Prusa/Orca	They have no plug-in API that can pick faces	A fake "plug-in" that is really a post-processing script: cannot pick

Decision	Why	Rejected
Real plug-in for Cura	Cura has a plug-in API with tools, picking and per-model settings	Same 3MF route for Cura: Cura's 3MF per-model-settings format was not investigated; the plug-in is the better fit anyway
Web UI (stdlib server + vendored three.js)	tkinter was a disaster on Bazzite; works the same on Linux/Windows/Mac; real 3D picking	tkinter software renderer: slow on old CPUs; PyQt: heavy
Pure-Python core, numpy optional	Runs on a 7th-gen i3 with nothing installed	numpy hard requirement
"Through the wall" by default	A filter wall that is open on one side only does not pass air	Fixed depth by default (still available)
Region sticks out 0.3 mm beyond the surface	A region that ends exactly on the surface can leave a skin film from rounding	Exactly flush
Auto prefers the cylinder if it covers more triangles	Fine STL tessellation (under 1° per facet) makes a cylinder look flat over a few facets	Auto = plane unless the flat patch is tiny (this was the first version and it failed at 3000 segments)

Tried and dropped / bugs found on the way

- Known Cura plug-in traps from earlier projects, checked here: (1) a tool-icon SVG with its own `fill` crashes Cura at start-up while the log still says "Loaded plugin": we ship no SVG and use Cura's built-in `MeshType` icon; (2) a flat copy into `plugins/` loads but is missing from Marketplace > Manage: now `plugins/<ID>/<ID>/` plus a `packages.json` record; (3) Cura hides the left toolbar until a model is loaded: Extensions menu entry and a one-time message; (4) "Loaded plugin" is weak proof: the check needs our self-test line with no crash text after it. All tested against a fake Cura folder and stand-in modules, **not a real Cura**; the `packages.json` record format is copied from the earlier plug-ins' notes, not re-verified here.
- First cylinder detector grew a patch of 400 triangles around the click; on a 12,500-segment cylinder that patch covered under 6° and the fit was rejected. Now the patch grows until the normals have turned ~30°.
- The web server did not read POST bodies it did not need, which broke the next request on the same keep-alive connection (`{}GET` seen as the method). It now always drains the body; a test uses one connection for the whole flow.
- Negative zero (`-0.00000`) in a 3MF transform: harmless, fixed for tidiness.

Measurements

Test machine: 2 cores, modern CPU, Python 3.13, one thread. **Not a 7th-gen Core i3**; expect a few times slower there. Benchmark: `tools/benchmark.py` (an example cartridge made with more facets).

Triangles	numpy	load STL	topology	pick	thickness	region	peak memory
48,000	yes	0.1 s	0.1 s	<0.1 s	<0.1 s	<0.1 s	89 MB
48,000	no	0.1 s	0.1 s	<0.1 s	0.1 s	<0.1 s	89 MB
200,000	yes	0.5 s	0.3 s	0.1 s	0.1 s	<0.1 s	275 MB (includes building the test mesh)

Triangles	numpy	load STL	topology	pick	thickness	region	peak memory
200,000	no	0.5 s	0.3 s	0.1 s	0.3 s	<0.1 s	275 MB

Server-side memory alone: about 810 bytes per triangle (200,000 triangles ≈ 200 MB); the loader refuses a model needing more than 60 % of free RAM (estimate 900 bytes per triangle) and says how to reduce it. The browser also needs memory for drawing.

Slicing the 3MF in PrusaSlicer 2.7.2 for the cartridge took a few seconds (not timed carefully).

Unknown / untested

- OrcaSlicer opening the Orca-flavour 3MF.
- Cura on Windows and Mac (on Linux Mint it works well in a real Cura, per the author's log). The icon reuses Cura's built-in `MeshType` icon and may look wrong.
- Windows and Mac launchers; Windows/Mac Cura folder detection.
- Pore size versus density and nozzle; air flow; whether carbon granules of a given size stay in; print quality of an open gyroid face on a bottom surface (first layer on the bed).
- Newer PrusaSlicer versions: setting names are from 2.7.2; a newer version may rename or retype some (e.g. enum values).
- A real 7th-gen Core i3.

Ideas

- **Mesh gyroid mode**: generate an actual gyroid solid (TPMS) as geometry, so pore size is exact and independent of the slicer. Heavy on triangles; needs marching cubes in C++.
- A **pore calibration coupon**: small test squares at several densities with your nozzle.
- A **housing and lid generator** (fill port, screw lid, gasket seat).
- Cones and free-form surfaces (pick by painting).
- SuperSlicer should read the PrusaSlicer-flavour 3MF; untested.
- A real window drawn on the face instead of sliders.

How to test a change

```
python3 tools/build_cura_plugin.py      # after touching
src/gyroid_filter/{mesh,regions,settings}.py
cd tests && python3 -W ignore -m unittest discover -p "test_*.py"
python3 tools/screenshots.py            # needs playwright + Chrome; fails on any
JavaScript error
```

`tests/test_prusaslicer.py` is skipped unless the `prusa-slicer` command exists (`sudo apt install prusa-slicer`).

Changelog

Changelog

Semantic versioning. Newest first.

0.1.0-wip: 2026-11-09

Logged 2026-11-09 19:56 (Europe/London), after the build above: - Cura plug-in tested by the author in a real Cura on Linux Mint: it works well. This is the first run in a real Cura; the earlier entries that say "not tested" for the Cura plug-in are superseded for Linux only. Windows and Mac Cura are still untested. - Cura versions, the exact steps, and the cutting-mesh results were not recorded in the report, so they are not claimed here.

First public work-in-progress build. Project started 2026-11-09 09:55.

Added - Core (`src/gyroid_filter`): STL and 3MF loading, flat-face and cylinder detection (including partial arcs and finely tessellated cylinders), wall-thickness probe, open-cell region solids for flat faces (whole, rectangle or oval window, rim) and cylinders (axial window, arc, rim), closed-solid check. - Web app (stdlib server, vendored three.js): open model, click to pick, live preview, regions list, exports (PrusaSlicer 3MF, OrcaSlicer 3MF, Region STLs zip), Cura plug-in installer tab that reads the files back and reads Cura's log. - Cura plug-in (`cura_plugin/GyroidFilter`): *Gyroid Filter* tool with panel, creates a cutting mesh with gyroid open-cell settings, re-apply, remove, start-up self-test in the log. - Cura installer hardening, checked against the bugs found in earlier Cura plug-ins (Hybrid Supports, SlicerSwap, Forge-X, Bambu LAN, LoK Walls): installs in Cura's own `plugins/<ID>/<ID>/` layout and records the plug-in in `packages.json` so it shows under Marketplace > Manage (an old flat copy is migrated); a `packages.json` that cannot be read is never overwritten and other plug-ins' records are kept; the log check needs the self-test line and no crash text after it, and says it cannot prove Cura stayed open; an Extensions menu entry and a one-time start-up message, because Cura hides its toolbar until a model is loaded; no SVG icon is shipped (an SVG that already had a `fill` attribute crashed Cura at start-up) and a test guards that; paths and log text are HTML-escaped in the installer tab. - Launchers for Linux, Windows and Mac; example cartridge; tests; PDF manual; GitHub Pages site; release workflow.

Tested for real: PrusaSlicer 2.7.2 slicing of the exported 3MF (G-code read back); web app in headless Chrome; Linux launcher; server API incl. security checks; geometry unit tests.

Not tested: OrcaSlicer opening the Orca 3MF; Cura on Windows and Mac (on Linux Mint it works well in a real Cura, per the author's log); Windows and Mac launchers; any real printing or filtering; a real 7th-gen Core i3.

Third Party

Third-party parts

The licence in LICENSE.md covers only Matthew Armstrong's own code. Everything below keeps its own licence. Nothing here implies any affiliation with or endorsement by these projects.

What	How it is used	Licence
three.js (r187, with its `OrbitControls`)	Bundled, minified, in `src/gyroid_filter/web/vendor/three-bundle.js` (made with esbuild, MIT). Licence text: `src/gyroid_filter/web/vendor/THREE-LICENSE.txt`	MIT
Cura and Uranium	Not included. The plug-in is written against their plug-in API; the `SupportEraser` tool was read as the model for the tool pattern, no code copied	LGPL-3.0
PrusaSlicer	Not included. Used only to test that its 3MF layout opens and slices (2.7.2 from the Ubuntu package)	AGPL-3.0
OrcaSlicer	Not included. Its 3MF importer and setting names were read to write a compatible file; no code copied	AGPL-3.0
PolyForm Noncommercial 1.0.0	The licence text in `LICENSE.md`	(the text of the licence itself)

Setting names and file layouts are facts about those programs' file formats, listed so the files can be read by them.

License

Licence

Required Notice: Copyright Matthew Armstrong (https://github.com/The-Dorkknight)

PolyForm Noncommercial License 1.0.0

https://polyformproject.org/licenses/noncommercial/1.0.0

Acceptance

In order to get any license under these terms, you must agree to them as both strict obligations and conditions to all your licenses.

Copyright License

The licensor grants you a copyright license for the software to do everything you might do with the software that would otherwise infringe the licensor's copyright in it for any permitted purpose. However, you may only distribute the software according to Distribution License and make changes or new works based on the software according to Changes and New Works License.

Distribution License

The licensor grants you an additional copyright license to distribute copies of the software. Your license to distribute covers distributing the software with changes and new works permitted by Changes and New Works License.

Notices

You must ensure that anyone who gets a copy of any part of the software from you also gets a copy of these terms or the URL for them above, as well as copies of any plain-text lines beginning with `Required Notice:` that the licensor provided with the software. For example:

> Required Notice: Copyright Yoyodyne, Inc. (http://example.com)

Changes and New Works License

The licensor grants you an additional copyright license to make changes and new works based on the software for any permitted purpose.

Patent License

The licensor grants you a patent license for the software that covers patent claims the licensor can license, or becomes able to license, that you would infringe by using the software.

Noncommercial Purposes

Any noncommercial purpose is a permitted purpose.

Personal Uses

Personal use for research, experiment, and testing for the benefit of public knowledge, personal study, private entertainment, hobby projects, amateur pursuits, or religious observance, without any anticipated commercial application, is use for a permitted purpose.

Noncommercial Organizations

Use by any charitable organization, educational institution, public research organization, public safety or health organization, environmental protection organization, or government institution is use for a permitted purpose regardless of the source of funding or obligations resulting from the funding.

Fair Use

You may have "fair use" rights for the software under the law. These terms do not limit them.

No Other Rights

These terms do not allow you to sublicense or transfer any of your licenses to anyone else, or prevent the licensor from granting licenses to anyone else. These terms do not imply any other licenses.

Patent Defense

If you make any written claim that the software infringes or contributes to infringement of any patent, your patent license for the software granted under these terms ends immediately. If your company makes such a claim, your patent license ends immediately for work on behalf of your company.

Violations

The first time you are notified in writing that you have violated any of these terms, or done anything with the software not covered by your licenses, your licenses can nonetheless continue if you come into full compliance with these terms, and take practical steps to correct past violations, within 32 days of receiving notice. Otherwise, all your licenses end immediately.

No Liability

As far as the law allows, the software comes as is, without any warranty or condition, and the licensor will not be liable to you for any damages arising out of these terms or the use or nature of the software, under any kind of legal claim.

Definitions

The **licensor** is the individual or entity offering these terms, and the **software** is the software the licensor makes available under these terms.

You refers to the individual or entity agreeing to these terms.

Your company is any legal entity, sole proprietorship, or other kind of organization that you work for, plus all organizations that have control over, are under the control of, or are under common control with that organization. **Control** means ownership of substantially all the assets of an entity, or the power to direct its management and policies by vote, contract, or otherwise. Control can be direct or indirect.

Your licenses are all the licenses granted to you for the software under these terms.

Use means anything you do with the software requiring one of your licenses.